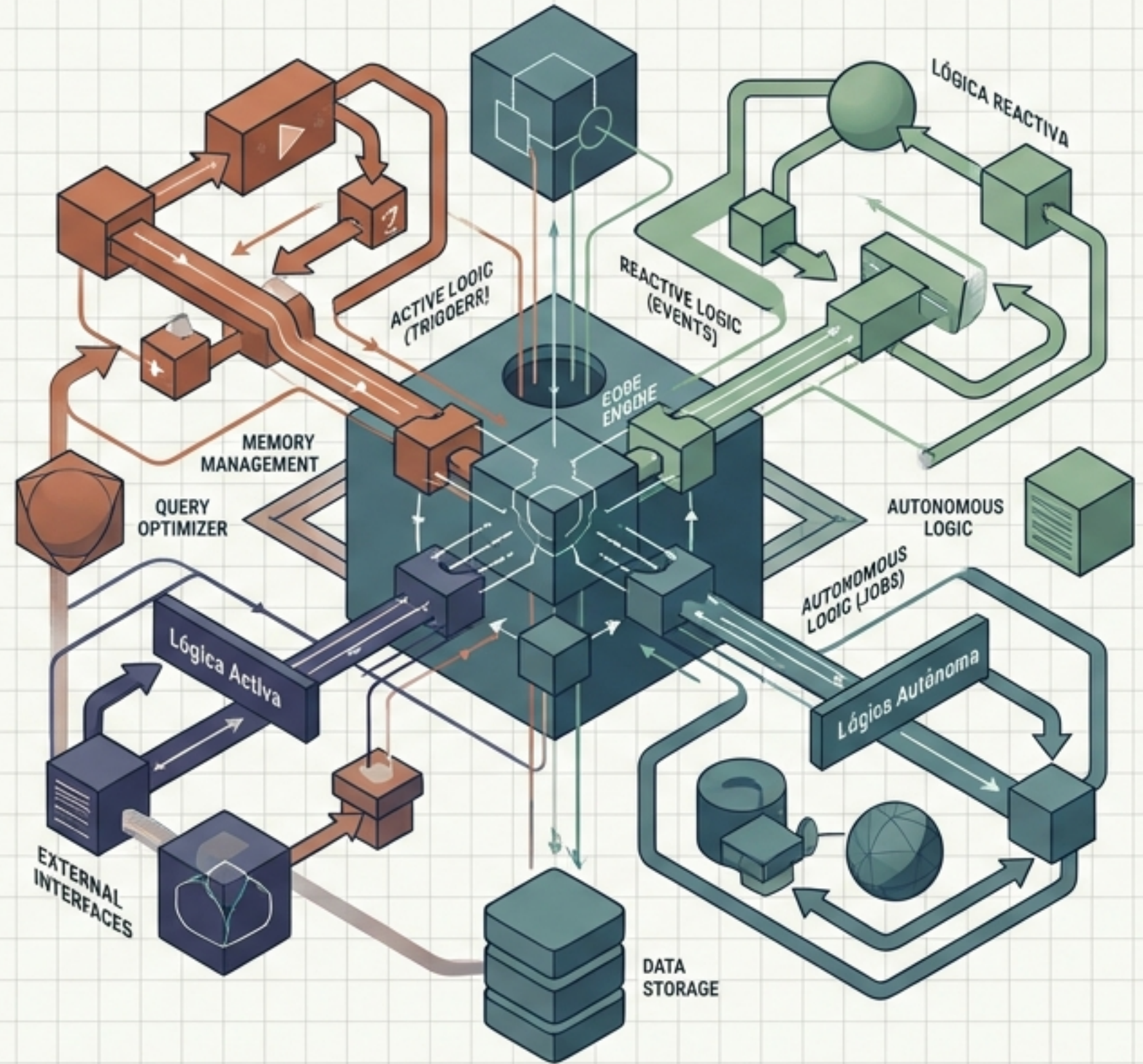
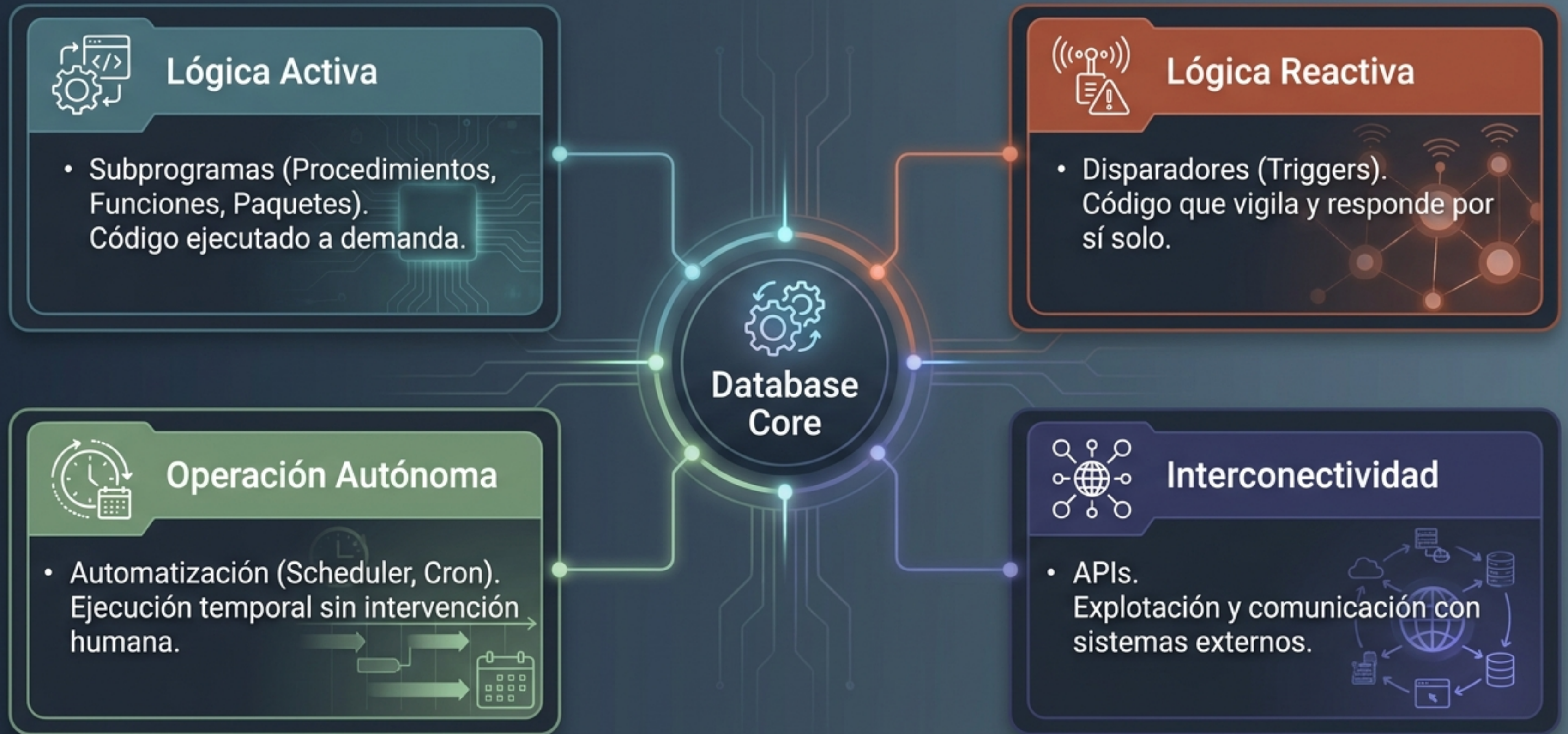


Arquitectura PL/SQL: El Motor de la Base de Datos

Dominando la lógica activa, reactiva y autónoma en entornos Oracle.

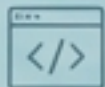


El Ecosistema PL/SQL



Matriz de Diagnóstico: Procedimientos vs. Funciones

Procedimientos	Funciones
<pre>CREATE [OR REPLACE] PROCEDURE nombreProcedimiento [(variable1 [modelo] tipoDatos, variable2 [modelo] tipoDatos...)] {IS AS} Declaraciones BEGIN ...</pre>	<pre>CREATE [OR REPLACE] FUNCTION nombreFuncion [(variable1 [modelo] tipoDatos, variable2 [modelo] tipoDatos...)] RETURN tipoDatos {IS AS} Declaraciones BEGIN ...</pre>
Propósito	Propósito
Procedimientos ejecutan acciones específicas.	Funciones realizan cálculos para devolver un valor.
Retorno (RETURN)	Retorno (RETURN)
Procedimientos no tienen un retorno obligatorio.	Funciones exigen la cláusula RETURN [tipo de dato].
Invocación	Invocación
Procedimientos usan EXECUTE o se llaman dentro de bloques anónimos.	Funciones se pueden integrar en sentencias SQL u otros subprogramas.



Capacidades Avanzadas Compartidas



Ambos soportan parámetros (modelos de datos), sobrecarga (mismo nombre, distintos parámetros) y recursividad.
Código fuente auditable en USER_SOURCE o mediante DESCRIBE.

Paquetes: Arquitectura Modular de Código



Capa Exterior (Especificación / AS)

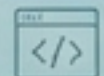
El escaparate público. Declara las variables, constantes, cursores, excepciones y cabeceras de subprogramas accesibles desde el exterior.

Núcleo Interior (Cuerpo / BODY)

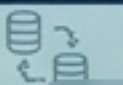
La sala de máquinas privada. Contiene la implementación funcional del código y esconde variables o subprogramas internos que no pueden ser invocados directamente desde fuera.

```
CREATE [OR REPLACE] PACKAGE nombrePaquete AS  
    [declaración de variables, constantes, cursores,  
    excepciones, funciones y procedimientos]  
END nombrePaquete;
```

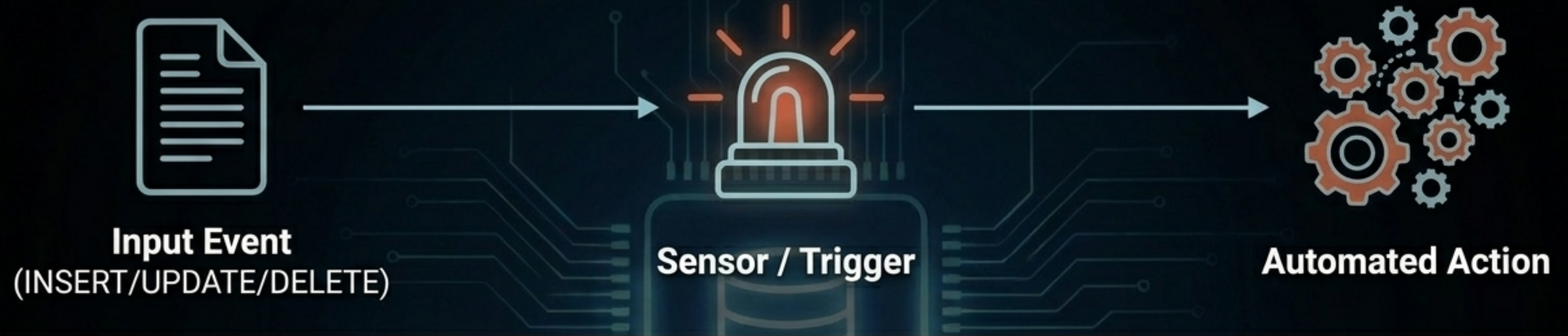
```
CREATE [OR REPLACE] PACKAGE BODY nombrePaquete AS  
    [declaración privadas y cuerpo de los  
    procedimientos y funciones]  
END nombrePaquete;
```



Nota de Invocación: Para llamar a un elemento externo, se utiliza la nomenclatura **nombrePaquete.subprograma()**.



Disparadores (Triggers): El Motor Reactivo



Definición Core: Un tipo de procedimiento que se ejecuta automáticamente ante una operación DML.
No requiere invocación explícita.

Auditoría

Rastrear cambios y registrar al usuario responsable.

Administración

Automatizar tareas de mantenimiento derivadas de una acción.

Integridad

Aplicar reglas de negocio complejas antes de consolidar datos.

Anatomía de un Disparador

Momento

{BEFORE | AFTER | INSTEAD}
} → Determina si se ejecuta antes, después, o en lugar de la instrucción DML.

Evento

{INSERT | UPDATE | DELETE} → La operación que activa la alarma.

```
CREATE [OR REPLACE] TRIGGER nombreDisparador
{BEFORE | AFTER | INSTEAD OF}
{INSERT [OR] UPDATE [OR] DELETE} ON Tabla
[REFERENCING OLD AS nomAntiguo NEW AS nomNuevo]
[FOR EACH ROW [WHEN (Condición de disparo)]]
BEGIN
    -- Bloque_de_código_del_Disparador
END nombreDisparador;
```

Alcance

FOR EACH ROW → Ejecución por cada fila afectada. Si se omite, se ejecuta una sola vez por instrucción (Nivel Instrucción).

Condición

WHEN → Filtro opcional (solo aplicable a nivel de fila) para restringir cuándo se dispara.

El Ciclo de Vida: Jerarquía de Ejecución

Contexto: Los disparadores asociados a una tabla no se ejecutan en paralelo. Existe un orden inquebrantable:



Memoria de Estado: Referencias OLD y NEW

Concepto: Durante una operación UPDATE, la base de datos retiene temporalmente ambas versiones del registro.

	\$100 Estado previo

:OLD.campo

Captura el estado previo. Fundamental para calcular deltas o registrar auditorías históricas.



	\$150 Estado propuesto

:NEW.campo

Captura el estado propuesto a insertar.

```
CREATE OR REPLACE TRIGGER auditoriaProductos
BEFORE UPDATE OF precio ON PRODUCTOS
FOR EACH ROW
WHEN (OLD.precio < NEW.precio)
BEGIN
    INSERT INTO AUDITORIA_PRODUCTOS
    VALUES (:OLD.precio, :NEW.precio, SYSDATE);
END;
```

Panel de Restricción

Estas referencias solo existen en disparadores de nivel de fila (FOR EACH ROW).

Disparadores Especiales e Inteligentes

Condicionales DML

Permite consolidar **múltiples eventos** en un solo bloque de código.

Identifica la operación exacta usando IF INSERTING, IF UPDATING, o IF DELETING devolviendo valores booleanos (TRUE).

```
CREATE OR REPLACE TRIGGER pruebaClausulas
BEFORE INSERT OR DELETE OR UPDATE OF campoPrueba ON tablaPrueba
FOR EACH ROW
BEGIN
    IF DELETING THEN
        --Instrucciones si el trigger se ejecuta al eliminar filas
    ELSIF INSERTING THEN
        --Instrucciones si el trigger se ejecuta al insertar filas
    ELSE
        --Instrucciones si el trigger se ejecuta al modificar filas
    END IF;
END;
```

INSTEAD OF

Herramienta **exclusiva** para **Vistas complejas** (compuestas por múltiples tablas mediante JOIN).

Intercepta un error de inserción redirigiendo los datos a sus tablas base correspondientes.

```
CREATE OR REPLACE TRIGGER insertarProducto
INSTEAD OF INSERT ON infoProductos
BEGIN
    INSERT INTO productos (modelo, precio)
    VALUES (:NEW.modelo, :NEW.precio);
    INSERT INTO almacen (modelo, precio, cantidad)
    VALUES (:NEW.modelo, :NEW.precio, :NEW.cantidad);
END;
```


Reglas de Oro y Gestión de Restricciones

Checklist de Seguridad

Operaciones Prohibidas (Alertas)



Prohibido el control de transacciones (COMMIT, ROLLBACK, SAVEPOINT).



Prohibidas variables de tipo LONG o LONG RAW.



Prohibida la invocación de subprogramas que realicen transacciones.

Comandos de Gestión

Borrado: DROP TRIGGER nombre;

Pausa/Reactivación: ALTER TRIGGER nombre {DISABLE | ENABLE};

Monitorización: Vista USER_TRIGGERS para metadatos del sistema.

```
DESC USER_TRIGGERS;
```

Nombre	¿Nulo?	Tipo
-----	-----	-----
TRIGGER_NAME		VARCHAR2(128)
TRIGGER_TYPE		VARCHAR2(16)
TRIGGERING_EVENT		VARCHAR2(246)
TABLE_OWNER		VARCHAR2(128)
BASE_OBJECT_TYPE		VARCHAR2(18)

Automatización Nivel OS: Cron y Shell Scripts

Shell Scripts + SQL*Plus

Encapsulación de comandos SQL dentro de scripts de Bash (.sh). Permite invocar sqlplus y ejecutar rutinas externas a la base de datos de manera automatizada.

```
#!/bin/bash
sqlplus username/password@database <<EOF
@/path/to/your/script.sql
EXIT;
EOF
```

Cron Jobs (Unix/Linux)

El demonio del sistema operativo. Permite programar la ejecución de los scripts shell mediante expresiones de calendario (ej. 0 5 * * * para ejecución diaria a las 5:00 AM).

```
0 5 * * * /scripts/job.sh
```


Automatización Nativa: Oracle Scheduler

Concepto: La evolución de DBMS_JOB. Planificación avanzada integrada en la base de datos.

```
BEGIN
  DBMS_SCHEDULER.create_job (
    job_name      => 'mi_job',
    job_type      => 'PLSQL_BLOCK',
    job_action    => 'BEGIN mi_procedimiento();
END;',
    start_date    => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY; BYHOUR=10;',
    enabled       => TRUE
  );
END;
```

Control Panel



Parámetros de create_job:

job_type: ¿Qué es? (ej. PLSQL_BLOCK).

job_action: ¿Qué hace? (El código a ejecutar).

start_date / repeat_interval: Frecuencia y calendario.

enabled: Activación inicial.

Botones de Control (Operaciones Auxiliares):

set_attribute

drop_job

run_job

stop_job

Ecosistema de Integración: Explotación y APIs

Oracle APEX

Desarrollo rápido (Low-code) para aplicaciones web y móviles responsivas.

Visual Builder

Plataforma cloud visual integrada al ecosistema Oracle.

Oracle Database

JDeveloper / Forms

Entornos IDE tradicionales para aplicaciones empresariales complejas (Java, XML).

Oracle API Catalog (OAC)

El centro de gobierno. Catálogo maestro para documentar, descubrir y centralizar el ciclo de vida de los servicios web corporativos.

ORACLE

FUSION MIDDLEWARE
API CATALOG

De la Teoría a la Práctica: Soluciones de Negocio

Estudio de Caso Tarjeta 1: Lógica Activa (Función)

El Reto

Identificar si un empleado cobra por encima de la **media** de su departamento.

La Solución

Una **Función** que recibe un **ID**, calcula promedios internamente y retorna un **BOOLEAN**. Manejo de excepciones **NO_DATA_FOUND** incorporado.

```
CREATE FUNCTION comprobarSalario(a NUMBER) RETURN BOOLEAN IS
  id_dpto EMPLEADOS.id_departamento%TYPE;
  sal EMPLEADOS.salario%TYPE;
  m_sal EMPLEADO.salario%TYPE;
BEGIN
  SELECT salario,id_empleado,id_departamento INTO sal,id_empl,id_dpto FROM EMPLEADOS
  WHERE id_empl = a;
  SELECT AVG(SALARIO) INTO m_sal FROM EMPLEADOS
  WHERE id_departamento = id_dpto;
  IF sal > m_sal THEN
    RETURN TRUE;
  ELSE
    RETURN FALSE;
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
```

PNN

Estudio de Caso Tarjeta 2: Lógica Reactiva (Trigger)

El Reto

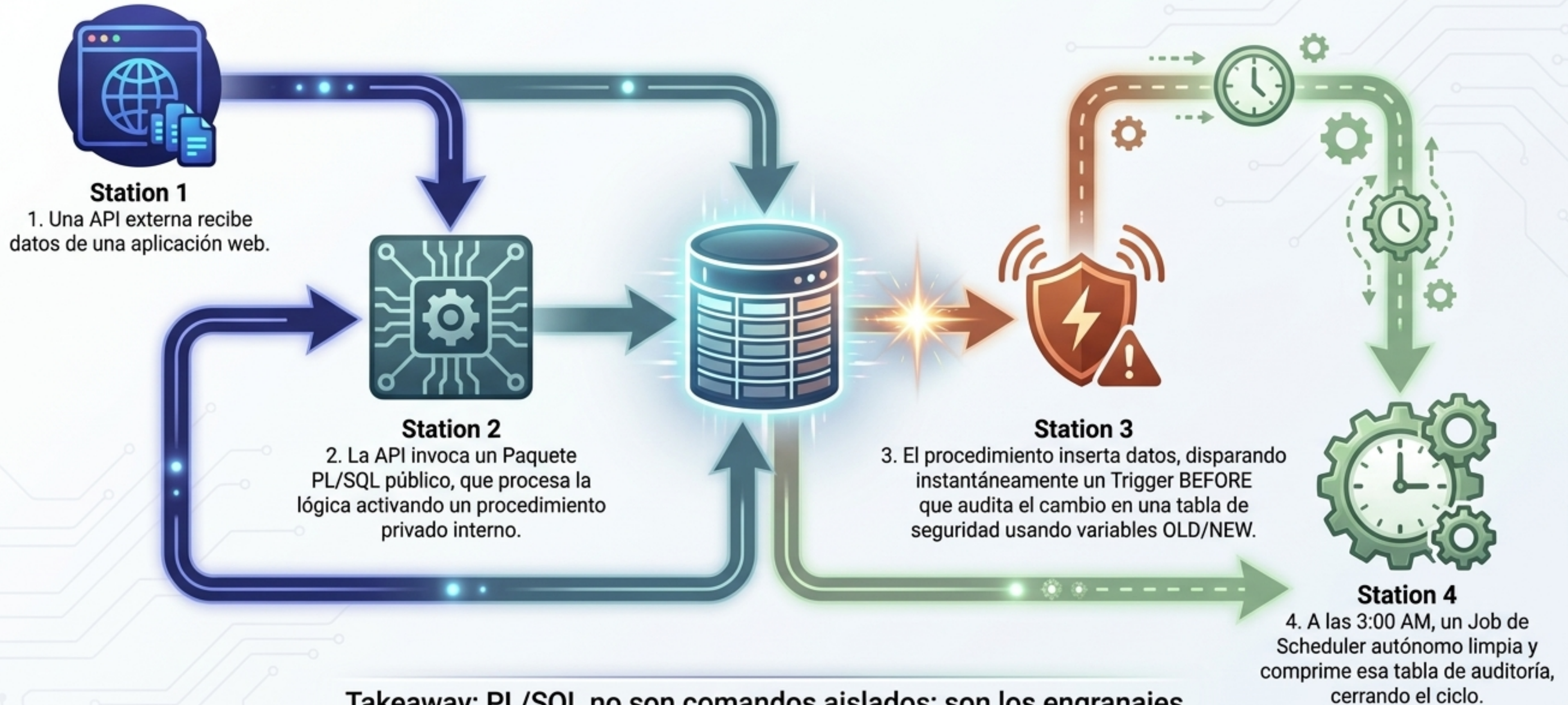
Evitar bases de datos **huérfanas bloqueando** el registro de empleados sin departamento.

La Solución

Trigger **BEFORE INSERT OR UPDATE**. Condicional **IF (:new.departamento IS NULL)** invoca **RAISE_APPLICATION_ERROR** para detener la transacción de inmediato.

```
CREATE OR REPLACE TRIGGER control_empleados
BEFORE INSERT OR UPDATE OF empleados
FOR EACH ROW
BEGIN
  IF (:new.departamento IS NULL) THEN
    RAISE_APPLICATION_ERROR(-20201, 'Todo
    empleado debe pertenecer a un departamento');
  END IF;
END;
```


El Blueprint: Un Ecosistema en Producción



Takeaway: PL/SQL no son comandos aislados; son los engranajes de un motor de datos vivo e interconectado.